

colony8

Transactional shared memory for multi-agent fleets.

Stateless agents. One memory. Every write resolved inside a serializable transaction.

THE FAILURE MODE

37% of multi-agent
failures

come from inconsistent shared
state

THE FIX

write-time resolution

not read-time ranking, not async
consolidation

THE PROOF

200 writes → 1 live fact

0 lost writes, race-test enforced

37%

of multi-agent system failures come from
agents acting on **inconsistent shared state**.

Cemri et al., failure-mode study – summarized by O'Reilly,
2025

HOW IT ACTUALLY GOES WRONG

agent A reads memory, starts a 4-second LLM call

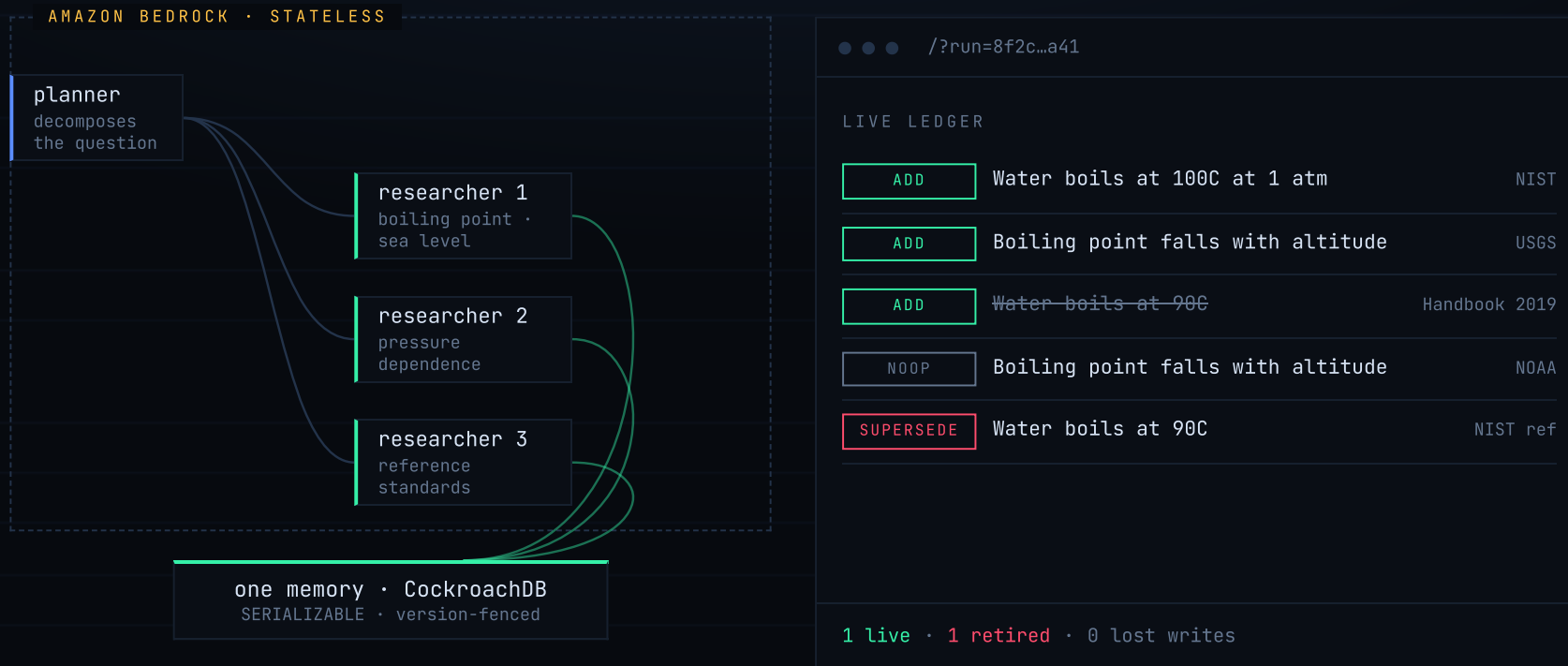
agent B writes a correction to the same fact

agent A finishes and writes over it – B's
correction is gone

No error is raised. Both agents proceed as if
nothing happened.

colony8 makes this impossible by
construction.

THE SHAPE OF IT • AGENTS DEPLOY, ONE MEMORY ABSORBS



Agents hold **zero state**. Every write lands in one transactional table.

Every candidate takes the same two phases

PHASE 1 · NO TRANSACTION, NO LOCK

recall
vector search, k=5



classify
ADD · UPDATE · NOOP · SUPERSEDE



PHASE 2 · ONE SERIALIZABLE TRANSACTION

SELECT ... FOR UPDATE
re-read the target row



verify + apply
version = snapshot?

version drift → re-snapshot

retries exhausted → DEFERRED

committed → 1 live fact + chain

```

# colony8/memory/resolver.py
for attempt in range(max_attempts):
    matches = recall(pool, run_id, embedding, k=5)
    if not matches or matches[0].similarity < ADD_THRESHOLD:
        decision = Decision(op="ADD", ...)
    else:
        decision = classify_fn(cand, matches)
    try:
        return _apply(pool, run_id, cand, ...)
    except (StaleSnapshot, SerializationFailure):
        time.sleep(random.uniform(0.01, 0.05) * (attempt + 1))
        continue

# attempts exhausted – parked, not dropped
log_event(conn, run_id, "DEFERRED", cand.title, ...)

```

```

# inside one SERIALIZABLE transaction
row = conn.execute(
    "SELECT version FROM findings "
    "WHERE id = %s AND invalidated_at IS NULL "
    "FOR UPDATE",
    (target.id,),
).fetchone()

if row is None or row[0] != target.version:
    raise StaleSnapshot(f"{target.id} moved")

# winner inserted, loser retired – never deleted
fid = insert_finding(conn, run_id, cand, embedding)
invalidate(conn, target.id, superseded_by=fid,
           expected_version=target.version)

```

Classification is expensive and happens **outside** the transaction. The transaction only re-reads, compares version, and applies.

The chain stays queryable forever

FINDING • THERMAL PROPERTIES OF WATER

RETIRED

~~Water boils at 90°C~~

Handbook, 2019

LIVE

Water boils at 100°C at 1 atm

NIST reference

```
-- the loser is marked, not removed
UPDATE findings
  SET invalidated_at = now(),
      superseded_by = %s,
      version        = version + 1
WHERE id = %s
AND invalidated_at IS NULL
AND version = %s
```

The ``version = %s`` predicate **is** the fence. If another agent moved the row since the snapshot, this update matches zero rows and the whole cycle re-runs against the new truth.

`resolution_events` keeps every decision the resolver ever made, with its reason – a complete audit log, not a bolt-on.

Kill the fleet mid-run. The memory doesn't care.

01 • CONTRADICTION LANDS

A 2019 handbook claims **90°C**.
Seconds later the NIST
reference arrives. The
resolver classifies it as a
contradiction and supersedes
the earlier claim on screen –
struck through, chain intact.

02 • BACKEND KILLED

Ctrl-C on camera, mid-run.
Connection refused. Every
agent in the fleet dies with
it.

There is nothing to lose – agents
hold zero state.

03 • FULL RECALL

Restart, reload with **?run=**
<id>. The entire ledger
replays from CockroachDB –
supersede chain and all.

| Nothing ever lived in agent state, so nothing was there to lose.

200 conflicting writes at one fact

CONTENTION RUN • SCRIPTS/BENCH.PY

1	live fact remaining
184	retired, with full audit chains
155	conflict retries absorbed
16	parked as DEFERRED – visible, retryable
0	lost writes

SUBSTRATE THROUGHPUT

OPERATION	THRU	P50/P95 MS
write, fast-path ADD	~135/s	7.6 / 8.3
recall, k=5	~395/s	2.3 / 3.7
contended SUPERSEDE ×8	~96/s	–

Single-node in-memory CockroachDB; LLM and embedding calls excluded – this measures the substrate, not model latency.

Numbers vary run to run. The invariants do not.

26 tests passing, including `test_concurrent_contradiction` – a real race, not a simulation.

Against typical agent-memory layers

	COLONY8	TYPICAL AGENT-MEMORY LAYERS*
conflict handling	resolved at write time, in a serializable transaction	read-time ranking, or async consolidation minutes-hours later
concurrent writers	version-fenced – one live fact, race-test proven	per-agent views can diverge over a shared archive
history	supersede chains – retired, never deleted	facts updated or overwritten in place
storage	SQL + vectors in one database, one transaction	separate vector + graph + KV stores stitched together
agent restart	full recall – agents hold zero state	framework session state to rebuild

* patterns documented across Mem0, Zep, and Letta/MemGPT public papers and docs.

REQUIRED TOOLS • EVERY ONE LOAD-BEARING

TOOL	ROLE ON THE DEMO PATH
CockroachDB Distributed Vector Indexing	semantic recall before every write, feeding candidates to the resolver
CockroachDB Cloud Managed MCP Server	the audit agent's only access path – read-only SQL over MCP
ccloud CLI	scripted, repeatable cluster + scoped SQL user provisioning
Amazon Bedrock	planner / researcher / classifier / audit LLM, and Titan v2 embeddings
AWS Lambda	stateless container runtime for the API and ledger replay
Amazon S3 + CloudFront	static hosting and CDN for the ledger UI

THE SHARPEST EDGE WE HIT

A vector index built with the default opclass is **silently ignored** by a query ordering on `↔` – no error, no warning, just a full scan. Adding an ordinary `WHERE` predicate to the ordering query defeats it the same silent way. `EXPLAIN` is the only way to find out.

THE INDEX IS ON THE HOT PATH – HERE IS THE PROOF

```
-- opclass must match the operator recall() orders by
CREATE VECTOR INDEX IF NOT EXISTS findings_embedding_idx
  ON findings (run_id, embedding vector_cosine_ops);
```

```
-- EXPLAIN, against 800 findings
• top-k
└─ • lookup join
    └─ • vector search
        table: findings@findings_embedding_idx
        target count: 40
```

TWO DETAILS THAT MAKE IT REAL

01 the opclass is `vector_cosine_ops`, matching the `↔` operator – an `l2` index falls back to a full scan without saying so.

02 `invalidated_at IS NULL` is not in the ordering query – that predicate defeats the index too. Recall over-fetches and filters retired rows afterwards.

If the index is ever unavailable, recall falls back to an exact scan – slower, identical results. Correctness never depends on it.

Failure modes are the product

WHAT GOES WRONG

Two agents write one fact at once

CockroachDB returns `SQLSTATE 40001`

All retries exhausted

A researcher agent crashes

The whole fleet dies

`EMBED_DIM` disagrees with the table

WHAT COLONY8 DOES

Loser's version check fails; re-snapshot and re-classify

Jittered retry, up to 5 attempts

DEFERRED event – parked and visible, never silently dropped

Subtopic marked failed; other agents' writes stay intact

Nothing to lose – the ledger replays from CockroachDB

Startup fails loudly, not one insert at a time

ONE HONEST GAP

Two agents concurrently `ADD` ing brand-new, near-identical claims can both land – there is no existing row yet to fence on. The duplicate isn't lost; it is retired the moment a later candidate matches and supersedes it.

COLONY8

One memory. Nothing lost.

Stateless agents on Bedrock, one CockroachDB table, every write resolved inside a serializable transaction — and a full audit chain behind every live fact.